Knowledge Graph 전처리 방법론

> **2025년 최신 트렌드 기반**
>
> 참고: Docs2KG, LLM-KG Survey, Neo4j Tutorial

---

1. Knowledge Graph 개요

1.1 Knowledge Graph란?

Knowledge Graph(지식 그래프)는 **엔티티(노드)**와 **관계(엣지)**를 구조화하여 지식을 표현하는 방식입니다.

```
(Apple) --[founded_by]--> (Steve Jobs)
   │                          │
   │                          └──[born_in]--> (San Francisco)
   │
   └──[headquartered_in]--> (Cupertino)
```

1.2 Triple 구조

```
(Subject) --[Predicate]--> (Object)
(주어)    --[관계]-->      (목적어)

예시:
("삼성전자", "설립연도", "1969")
("삼성전자", "본사위치", "수원시")
("이재용", "직책", "회장")
("이재용", "소속", "삼성전자")
```

1.3 Knowledge Graph 유형

유형	설명	예시
Layout KG	문서 구조/레이아웃 표현	헤딩, 섹션, 페이지 계층
Metadata KG	문서 메타데이터	저자, 생성일, 키워드
Semantic KG	도메인 지식, 엔티티/관계	인물, 조직, 사건, 인과관계

---

2. KG 구축 파이프라인

2.1 전체 아키텍처

```
┌──────────────────────────────────────────────────────────────────────┐
│                      Knowledge Graph Pipeline                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [문서]                                                              │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────────────┐                                                    │
│  │ 1. 문서 파싱  │  PDF, DOCX, HTML → 구조화된 텍스트                 │
│  └──────────────┘                                                    │
│         │                                                            │
│         ▼                                                            │
│  ┌──────────────┐                                                    │
│  │ 2. 엔티티 추출│  NER: 인물, 조직, 장소, 날짜 등                    │
│  │    (NER)     │                                                    │
│  └──────────────┘                                                    │
│         │                                                            │
│         ▼                                                            │
│  ┌──────────────┐                                                    │
│  │ 3. 관계 추출 │  RE: Subject-Predicate-Object 트리플               │
│  │    (RE)      │                                                    │
│  └──────────────┘                                                    │
│         │                                                            │
│         ▼                                                            │
│  ┌──────────────┐                                                    │
│  │ 4. 온톨로지  │  스키마 정의, 클래스/속성 매핑                      │
│  │    매핑      │                                                    │
│  └──────────────┘                                                    │
│         │                                                            │
│         ▼                                                            │
│  ┌──────────────┐                                                    │
│  │ 5. 그래프 DB │  Neo4j, Amazon Neptune, Stardog                    │
│  │    저장      │                                                    │
│  └──────────────┘                                                    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

3. Named Entity Recognition (NER)

3.1 엔티티 유형

유형	태그	예시
인물	PER	이재용, 팀 쿡
조직	ORG	삼성전자, Apple
장소	LOC	서울, 쿠퍼티노
날짜	DATE	2024년 12월

유형	태그	예시
금액	MONEY	100억원
제품	PRODUCT	갤럭시 S24
이벤트	EVENT	CES 2025

3.2 NER 도구

GLiNER (Generalist Lightweight NER)

```
from gliner import GLiNER

model = GLiNER.from_pretrained("urchade/gliner_multi-v2.1")

text = "삼성전자 이재용 회장이 2024년 CES에서 AI 전략을 발표했다."

labels = ["인물", "조직", "날짜", "이벤트"]
entities = model.predict_entities(text, labels)

# 결과:
# [
#   {"text": "삼성전자", "label": "조직", "start": 0, "end": 4},
#   {"text": "이재용", "label": "인물", "start": 5, "end": 8},
#   {"text": "2024년", "label": "날짜", "start": 13, "end": 18},
#   {"text": "CES", "label": "이벤트", "start": 19, "end": 22}
# ]
```

SpaCy + 한국어

```
import spacy

# 한국어 모델
nlp = spacy.load("ko_core_news_lg")

doc = nlp("삼성전자가 서울에서 신제품을 발표했다.")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

LLM 기반 NER

```
PROMPT = """
다음 텍스트에서 엔티티를 추출하세요.

텍스트: {text}
```

```
JSON 형식으로 출력:
{{"entities": [
    {{"text": "...", "type": "PERSON|ORG|LOC|DATE|..."}}
]}}
"""

entities = llm.generate(PROMPT.format(text=text))
```

---

4. Relation Extraction (RE)

4.1 관계 유형

관계	설명	예시
`founded_by`	설립자	(삼성, founded_by, 이병철)
`located_in`	위치	(삼성전자, located_in, 수원)
`subsidiary_of`	자회사	(삼성SDI, subsidiary_of, 삼성)
`ceo_of`	CEO	(이재용, ceo_of, 삼성전자)
`occurred_on`	발생일	(발표, occurred_on, 2024-01-09)
`caused_by`	원인	(주가하락, caused_by, 실적악화)

4.2 RE 방법론

규칙 기반 (Dependency Parsing)

```python
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Steve Jobs founded Apple in 1976.")

# 의존 구문 분석
for token in doc:
    if token.dep_ == "nsubj" and token.head.pos_ == "VERB":
        subject = token.text
        verb = token.head.text
        # Object 찾기
        for child in token.head.children:
            if child.dep_ == "dobj":
                obj = child.text
                print(f"({subject}, {verb}, {obj})")
# 출력: (Jobs, founded, Apple)
```

LLM 기반 RE

```
PROMPT = """
다음 텍스트에서 엔티티 간 관계를 추출하세요.

텍스트: {text}

엔티티: {entities}

JSON 형식으로 관계 트리플 출력:
{{"triples": [
    {{"subject": "...", "predicate": "...", "object": "..."}}
]}}
"""
```

OpenIE (Open Information Extraction)

```
from openie import StanfordOpenIE

with StanfordOpenIE() as client:
    text = "삼성전자 이재용 회장이 CES 2024에서 AI 비전을 발표했다."
    triples = client.annotate(text)

# [
#   {"subject": "이재용", "relation": "회장", "object": "삼성전자"},
#   {"subject": "이재용", "relation": "발표했다", "object": "AI 비전"},
# ]
```

---

5. 온톨로지 구축

5.1 온톨로지란?

온톨로지는 도메인의 **개념 체계**와 **관계 스키마**를 정의합니다.

```
┌──────────────────────────────────────────┐
│              Ontology                    │
├──────────────────────────────────────────┤
│  Classes (클래스)                        │
│  ├─ Organization                         │
│  │   ├─ Company                          │
│  │   └─ Government                       │
│  ├─ Person                               │
│  │   ├─ Executive                        │
│  │   └─ Employee                         │
│  └─ Location                             │
│      ├─ Country                          │
│      └─ City                             │
│                                          │
│  Properties (속성)                       │
```

```
│   ├─ founded_by: Company → Person          │
│   ├─ located_in: Organization → Location   │
│   └─ works_for: Person → Organization      │
└──────────────────────────────────────────┘
```

5.2 OWL 온톨로지 정의

```
<!-- company_ontology.owl -->
<owl:Class rdf:about="#Company">
    <rdfs:subClassOf rdf:resource="#Organization"/>
</owl:Class>

<owl:ObjectProperty rdf:about="#foundedBy">
    <rdfs:domain rdf:resource="#Company"/>
    <rdfs:range rdf:resource="#Person"/>
</owl:ObjectProperty>

<owl:DatatypeProperty rdf:about="#foundedYear">
    <rdfs:domain rdf:resource="#Company"/>
    <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>
```

5.3 LLM 기반 온톨로지 생성

```
ONTOLOGY_PROMPT = """
다음 도메인에 대한 온톨로지를 설계하세요: {domain}

요구사항:
1. 주요 클래스(개념) 정의
2. 클래스 간 계층 구조
3. 관계(Object Properties) 정의
4. 데이터 속성(Datatype Properties) 정의

JSON-LD 형식으로 출력:
"""

ontology = llm.generate(ONTOLOGY_PROMPT.format(domain="기업 재무"))
```

---

6. Docs2KG 프레임워크

6.1 개요

Docs2KG는 Human-LLM 협업 방식의 KG 구축 프레임워크입니다.

```
```

```
┌──────────────────────────────────────────────────────────────────┐
│                          Docs2KG                                 │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [이기종 문서]                                                    │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────────┐       │
│  │              Multi-faceted KG                        │       │
│  │  ┌────────────┬────────────┬────────────┐            │       │
│  │  │ Layout KG  │Metadata KG│Semantic KG│            │       │
│  │  │            │            │            │            │       │
│  │  │ 문서구조   │ 메타데이터│ 도메인     │            │       │
│  │  │ 계층       │ 속성       │ 엔티티/    │            │       │
│  │  │            │            │ 관계       │            │       │
│  │  └────────────┴────────────┴────────────┘            │       │
│  └──────────────────────────────────────────────────────┘       │
│                                                                  │
│  Construction Paradigms:                                         │
│  1. Ontology-based (기존 스키마 활용)                             │
│  2. Hybrid NLP + LLM (NER/RE + LLM 검증)                         │
│  3. LLM-guided ontology (LLM이 스키마 생성)                       │
│  4. Specialized models (특화 모델)                                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

6.2 Layout KG

문서의 물리적 구조를 그래프로 표현:

```
// Neo4j Cypher
CREATE (doc:Document {name: "보고서.pdf"})
CREATE (sec1:Section {title: "1. 개요", level: 1})
CREATE (sec2:Section {title: "1.1 배경", level: 2})
CREATE (para:Paragraph {text: "..."})
CREATE (table:Table {rows: 5, cols: 3})

CREATE (doc)-[:HAS_SECTION]->(sec1)
CREATE (sec1)-[:HAS_SUBSECTION]->(sec2)
CREATE (sec2)-[:CONTAINS]->(para)
CREATE (sec2)-[:CONTAINS]->(table)
```

6.3 Semantic KG

도메인 지식을 그래프로 표현:

```
// 엔티티 생성
CREATE (samsung:Company {name: "삼성전자", founded: 1969})
```

```
CREATE (jay:Person {name: "이재용", title: "회장"})
CREATE (suwon:City {name: "수원시"})

// 관계 생성
CREATE (jay)-[:CEO_OF]->(samsung)
CREATE (samsung)-[:HEADQUARTERED_IN]->(suwon)
```

---

7. GraphRAG

7.1 개념

GraphRAG는 Knowledge Graph와 RAG를 결합하여 검색 정확도를 높이는 방식입니다.

```
┌──────────────────────────────────────────────────────────────────┐
│                          GraphRAG                                 │
├──────────────────────────────────────────────────────────────────┤
│                                                                   │
│  [질문] "삼성전자 회장은 누구이며, 본사는 어디인가?"              │
│          │                                                        │
│     ┌────┴────┐                                                   │
│     ▼         ▼                                                   │
│  [NER]      [Intent]                                              │
│    │          │                                                   │
│    ▼          ▼                                                   │
│  "삼성전자"  "회장", "본사"                                       │
│    │                                                              │
│    ▼                                                              │
│  ┌──────────────────────────────────────┐                         │
│  │           Knowledge Graph            │                         │
│  │                                      │                         │
│  │  (삼성전자)──[CEO_OF]──>(이재용)      │                         │
│  │      │                               │                         │
│  │      └──[LOCATED_IN]──>(수원)         │                         │
│  │                                      │                         │
│  └──────────────────────────────────────┘                         │
│                   │                                               │
│                   ▼                                               │
│  [컨텍스트] + [LLM] → "삼성전자 회장은 이재용이며,                 │
│                        본사는 수원에 위치합니다."                  │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

7.2 GraphRAG 구현

```python
from neo4j import GraphDatabase

class GraphRAG:
    def __init__(self, uri, user, password):
```

```
        self.driver = GraphDatabase.driver(uri, auth=(user, password))

    def retrieve(self, query):
        # 1. 질문에서 엔티티 추출
        entities = extract_entities(query)

        # 2. 관련 서브그래프 검색
        with self.driver.session() as session:
            cypher = """
            MATCH (e:Entity)-[r]-(related)
            WHERE e.name IN $entities
            RETURN e, r, related
            LIMIT 50
            """
            result = session.run(cypher, entities=entities)

            # 그래프 컨텍스트 구성
            context = self._format_graph_context(result)

        return context

    def generate(self, query, context):
        prompt = f"""
        다음 지식 그래프 정보를 바탕으로 질문에 답하세요:

        지식 그래프:
        {context}

        질문: {query}
        """
        return llm.generate(prompt)
```

---

8. 이벤트/인과관계 추출

8.1 이벤트 추출

```
EVENT_PROMPT = """
텍스트에서 이벤트를 추출하세요:

텍스트: {text}

이벤트 구조:
- trigger: 이벤트를 나타내는 단어
- type: 이벤트 유형 (발표, 인수, 출시, 소송 등)
- arguments:
  - agent: 행위자
  - patient: 대상
  - time: 시간
  - location: 장소
```

```
JSON 형식으로 출력:
"""

# 예시 입력: "삼성전자가 2024년 1월 CES에서 새로운 AI TV를 발표했다."
# 출력:
# {
#   "trigger": "발표했다",
#   "type": "발표",
#   "arguments": {
#     "agent": "삼성전자",
#     "patient": "AI TV",
#     "time": "2024년 1월",
#     "location": "CES"
#   }
# }
```

8.2 인과관계 추출

```
CAUSAL_PROMPT = """
텍스트에서 인과관계를 추출하세요:

텍스트: {text}

형식:
- cause: 원인
- effect: 결과
- relation: 관계 유형 (causes, leads_to, results_in)

JSON으로 출력:
"""

# 예시: "반도체 수요 급증으로 삼성전자 주가가 상승했다."
# 출력:
# {
#   "cause": "반도체 수요 급증",
#   "effect": "삼성전자 주가 상승",
#   "relation": "causes"
# }
```

---

9. 도구 및 라이브러리

도구	용도	설치
Neo4j	그래프 데이터베이스	neo4j.com
GLiNER	엔티티 추출	`pip install gliner`
SpaCy	NLP 파이프라인	`pip install spacy`

도구	용도	설치
Owlready2	온톨로지 처리	`pip install owlready2`
RDFLib	RDF 처리	`pip install rdflib`
LangChain	LLM 통합	`pip install langchain`

---

10. 전처리 체크리스트

KG 구축 체크리스트

- ☐ 도메인 분석 및 온톨로지 설계
- ☐ 문서 파싱 및 구조 추출
- ☐ 엔티티 추출 (NER)
- ☐ 관계 추출 (RE)
- ☐ 온톨로지 매핑
- ☐ 트리플 검증 및 정제
- ☐ 그래프 DB 저장
- ☐ 인덱싱 및 쿼리 최적화

품질 체크리스트

- ☐ 엔티티 정규화 (동의어, 약어 처리)
- ☐ 관계 일관성 검증
- ☐ 순환 참조 체크
- ☐ 스키마 준수 여부 확인
- ☐ Human-in-the-loop 검증

---

11. Microsoft GraphRAG 심층 분석

11.1 GraphRAG 아키텍처

Microsoft GraphRAG는 기존 RAG의 한계(전역 질문 처리 어려움)를 해결하기 위해 Knowledge Graph + 커뮤니티 요약을 결합합니다.

```
┌──────────────────────────────────────────────────────────────────┐
│                    Microsoft GraphRAG Pipeline                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [문서 코퍼스]                                                   │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────────┐       │
│  │ 1. Indexing Pipeline                                 │       │
│  │    ├─ 텍스트 청킹 (300 토큰)                          │       │
│  │    ├─ LLM 기반 엔티티/관계 추출                       │       │
│  │    ├─ 엔티티 요약 생성                                │       │
│  │    ├─ 그래프 커뮤니티 탐지 (Leiden 알고리즘)          │       │
```

```
│  │    └─ 커뮤니티별 요약 생성 (계층적)                    │   │
│  └──────────────────────────────────────────────────────┘   │
│       │                                                      │
│       ▼                                                      │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ 2. Knowledge Graph + Community Summaries              │   │
│  │                                                       │   │
│  │    Communities:     Level 0  Level 1  Level 2         │   │
│  │                        ●        ●        ●             │   │
│  │                       /|\      / \      / \            │   │
│  │                      ● ● ●    ●   ●    ●   ●           │   │
│  │                                                       │   │
│  └──────────────────────────────────────────────────────┘   │
│       │                                                      │
│       ▼                                                      │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ 3. Query Processing                                   │   │
│  │    ├─ Local Search: 특정 엔티티 중심 검색              │   │
│  │    └─ Global Search: 커뮤니티 요약 기반 전역 검색      │   │
│  └──────────────────────────────────────────────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

11.2 GraphRAG 설치 및 설정

```
# 설치
pip install graphrag

# 프로젝트 초기화
graphrag init --root ./my_project

# 디렉토리 구조
# my_project/
# ├── settings.yaml      # 설정 파일
# ├── prompts/           # 커스텀 프롬프트
# └── input/             # 입력 문서
```

settings.yaml 예시:

```
# settings.yaml
encoding_model: cl100k_base
llm:
  api_key: ${OPENAI_API_KEY}
  model: gpt-4o-mini
  model_supports_json: true

embeddings:
  llm:
    api_key: ${OPENAI_API_KEY}
```

```yaml
    model: text-embedding-3-small

chunks:
  size: 300
  overlap: 100

entity_extraction:
  max_gleanings: 1
  prompt: prompts/entity_extraction.txt

claim_extraction:
  enabled: true
  description: "주요 주장과 사실을 추출합니다"

community_reports:
  max_length: 2000
  prompt: prompts/community_report.txt

cluster_graph:
  max_cluster_size: 10
```

11.3 인덱싱 실행

```bash
# 인덱싱 실행 (모든 문서 처리)
graphrag index --root ./my_project

# 실제 출력 예시:
# ⠋ Loading input documents...
# ✓ Loaded 150 documents
# ⠋ Chunking documents...
# ✓ Created 2,450 chunks
# ⠋ Extracting entities and relationships...
# ✓ Extracted 1,234 entities, 3,456 relationships
# ⠋ Generating entity summaries...
# ✓ Generated 1,234 entity summaries
# ⠋ Detecting communities (Leiden)...
# ✓ Found 45 communities across 3 levels
# ⠋ Generating community reports...
# ✓ Generated 45 community reports
# ✓ Indexing complete!
```

11.4 쿼리 실행

```python
import asyncio
from graphrag.query.llm.oai.chat_openai import ChatOpenAI
from graphrag.query.llm.oai.typing import OpenaiApiType
from graphrag.query.structured_search.local_search.mixed_context import
LocalSearchMixedContext
from graphrag.query.structured_search.global_search.search import GlobalSearch
```

```python
# LLM 설정
llm = ChatOpenAI(
    api_key="your-api-key",
    model="gpt-4o-mini",
    api_type=OpenaiApiType.OpenAI,
)

# === Local Search (특정 엔티티 검색) ===
async def local_search(query: str):
    """로컬 검색: 특정 엔티티와 관련된 정보 검색"""
    context_builder = LocalSearchMixedContext(
        community_reports=community_reports,
        text_units=text_units,
        entities=entities,
        relationships=relationships,
    )

    search_engine = LocalSearch(
        llm=llm,
        context_builder=context_builder,
        token_encoder=token_encoder,
        response_type="multiple paragraphs",
    )

    result = await search_engine.asearch(query)
    return result

# === Global Search (전역 질문) ===
async def global_search(query: str):
    """글로벌 검색: 전체 데이터셋에 대한 질문"""
    context_builder = GlobalCommunityContext(
        community_reports=community_reports,
        entities=entities,
    )

    search_engine = GlobalSearch(
        llm=llm,
        context_builder=context_builder,
        token_encoder=token_encoder,
        max_data_tokens=12000,
        response_type="multiple paragraphs",
    )

    result = await search_engine.asearch(query)
    return result


# 실행 예시
async def main():
    # Local Search - 특정 엔티티 관련 질문
    local_result = await local_search("삼성전자의 주요 제품은 무엇인가?")
    print("=== Local Search 결과 ===")
    print(local_result.response)
```

```
    print(f"참조 엔티티: {[e.title for e in
local_result.context_data['entities']]}")

    # Global Search - 전체 문서에 대한 질문
    global_result = await global_search("이 문서들의 주요 테마는 무엇인가?")
    print("\n=== Global Search 결과 ===")
    print(global_result.response)
    print(f"사용된 커뮤니티: {len(global_result.context_data['reports'])}")

asyncio.run(main())
```

실제 출력 예시:

```
=== Local Search 결과 ===
삼성전자의 주요 제품은 다음과 같습니다:

1. **반도체 (Semiconductor)**
   - DRAM, NAND Flash 메모리
   - 시스템 LSI, 파운드리 서비스

2. **스마트폰 (Mobile)**
   - Galaxy S 시리즈, Galaxy Z Fold/Flip
   - Galaxy A 시리즈 (중저가)

3. **가전제품 (Consumer Electronics)**
   - QLED/Neo QLED TV, 비스포크 냉장고
   - 세탁기, 에어컨

참조 엔티티: ['삼성전자', 'Galaxy S24', 'DRAM', 'Neo QLED', '이재용']

=== Global Search 결과 ===
이 문서들의 주요 테마는 크게 3가지로 분류됩니다:

1. **기업 경영 및 리더십** (45% 관련도)
   - 주요 기업들의 경영 전략과 CEO 리더십
   - 글로벌 시장 확장 전략

2. **기술 혁신** (35% 관련도)
   - AI/반도체 기술 발전
   - 차세대 디스플레이 기술

3. **ESG 및 지속가능성** (20% 관련도)
   - 탄소중립 목표 및 이행
   - 사회적 책임 활동

사용된 커뮤니티: 12
```

11.5 Leiden 커뮤니티 탐지 알고리즘

```python
import networkx as nx
from graspologic.partition import hierarchical_leiden
import numpy as np

def detect_communities(entities: list, relationships: list) -> dict:
    """
    Leiden 알고리즘으로 그래프 커뮤니티 탐지

    Returns:
        {
            "communities": {
                0: {"entities": [...], "level": 0},
                1: {"entities": [...], "level": 0},
                ...
            },
            "hierarchy": {...}
        }
    """
    # 그래프 생성
    G = nx.Graph()

    # 노드 추가
    for entity in entities:
        G.add_node(entity["id"], **entity)

    # 엣지 추가
    for rel in relationships:
        G.add_edge(
            rel["source"],
            rel["target"],
            weight=rel.get("weight", 1.0),
            description=rel.get("description", "")
        )

    # Leiden 알고리즘 실행 (계층적)
    community_mapping = hierarchical_leiden(
        G,
        max_cluster_size=10,  # 최대 커뮤니티 크기
        random_seed=42
    )

    # 결과 정리
    communities = {}
    for node_id, community_id in community_mapping.items():
        if community_id not in communities:
            communities[community_id] = {
                "entities": [],
                "level": 0  # 레벨 계산 로직 추가 필요
            }
        communities[community_id]["entities"].append(node_id)

    return {
        "communities": communities,
```

```
        "total_communities": len(communities),
        "modularity": nx.community.modularity(
            G, [set(c["entities"]) for c in communities.values()]
        )
    }


# 실제 사용 예시
entities = [
    {"id": "삼성전자", "type": "ORG"},
    {"id": "이재용", "type": "PERSON"},
    {"id": "SK하이닉스", "type": "ORG"},
    {"id": "최태원", "type": "PERSON"},
    # ...
]

relationships = [
    {"source": "이재용", "target": "삼성전자", "type": "CEO_OF", "weight": 1.0},
    {"source": "최태원", "target": "SK하이닉스", "type": "CEO_OF", "weight": 1.0},
    {"source": "삼성전자", "target": "SK하이닉스", "type": "COMPETITOR", "weight":
0.5},
    # ...
]

result = detect_communities(entities, relationships)
print(f"탐지된 커뮤니티 수: {result['total_communities']}")
print(f"Modularity: {result['modularity']:.3f}")

# 출력:
# 탐지된 커뮤니티 수: 5
# Modularity: 0.734
```

11.6 커뮤니티 요약 생성

```
from typing import List, Dict

COMMUNITY_REPORT_PROMPT = """
당신은 지식 그래프에서 추출한 커뮤니티 정보를 요약하는 전문가입니다.

## 커뮤니티 정보

### 엔티티 목록
{entities}

### 관계 목록
{relationships}

### 관련 텍스트 청크
{text_chunks}

## 작성 지침
```

```
1. 이 커뮤니티의 주요 테마를 파악하세요
2. 핵심 엔티티와 그들의 역할을 설명하세요
3. 주요 관계와 상호작용을 설명하세요
4. 이 커뮤니티가 나타내는 도메인/주제를 명시하세요

## 커뮤니티 요약 (2000자 이내)
"""

async def generate_community_report(
    community_id: int,
    entities: List[Dict],
    relationships: List[Dict],
    text_chunks: List[str],
    llm
) -> Dict:
    """커뮤니티 요약 보고서 생성"""

    # 엔티티 포맷팅
    entity_text = "\n".join([
        f"- {e['name']} ({e['type']}): {e.get('description', 'N/A')}"
        for e in entities
    ])

    # 관계 포맷팅
    rel_text = "\n".join([
        f"- {r['source']} --[{r['type']}]--> {r['target']}: {r.get('description',
'')}"
        for r in relationships
    ])

    # 텍스트 청크 포맷팅
    chunks_text = "\n---\n".join(text_chunks[:5])  # 상위 5개만

    prompt = COMMUNITY_REPORT_PROMPT.format(
        entities=entity_text,
        relationships=rel_text,
        text_chunks=chunks_text
    )

    response = await llm.generate(prompt)

    return {
        "community_id": community_id,
        "title": extract_title(response),
        "summary": response,
        "rating": calculate_importance(entities, relationships),
        "entity_count": len(entities),
        "relationship_count": len(relationships)
    }


# 실제 생성된 커뮤니티 리포트 예시
example_report = {
    "community_id": 3,
```

```
    "title": "한국 반도체 산업 생태계",
    "summary": """
    ## 커뮤니티 개요
    이 커뮤니티는 한국의 주요 반도체 기업들과 관련 인물, 기술을 포함합니다.

    ## 핵심 엔티티
    - **삼성전자**: 글로벌 1위 메모리 반도체 제조사. DRAM, NAND Flash 시장 주도.
    - **SK하이닉스**: 글로벌 2위 메모리 반도체 기업. HBM(고대역폭 메모리) 강점.
    - **이재용**: 삼성전자 회장, 반도체 투자 확대 주도
    - **최태원**: SK그룹 회장, AI 반도체 투자 결정

    ## 주요 관계
    - 삼성전자 ↔ SK하이닉스: 경쟁 관계 (메모리 시장)
    - 삼성전자 → NVIDIA: 파트너십 (HBM 공급)
    - SK하이닉스 → NVIDIA: 파트너십 (HBM 공급)

    ## 핵심 테마
    1. AI 반도체 시장 경쟁 심화
    2. HBM 기술 주도권 경쟁
    3. 글로벌 공급망 재편

    ## 중요도
    이 커뮤니티는 전체 문서에서 매우 높은 중요도를 가지며,
    한국 경제와 글로벌 기술 산업의 핵심을 나타냅니다.
    """,
    "rating": 9.5,
    "entity_count": 15,
    "relationship_count": 23
}
```

---

12. 전체 KG 파이프라인 구현

12.1 통합 파이프라인

```python
from dataclasses import dataclass, field
from typing import List, Dict, Optional, Set
from enum import Enum
import json
import hashlib

class EntityType(Enum):
    PERSON = "PERSON"
    ORGANIZATION = "ORG"
    LOCATION = "LOC"
    DATE = "DATE"
    PRODUCT = "PRODUCT"
    EVENT = "EVENT"
    CONCEPT = "CONCEPT"

@dataclass
```

```python
class Entity:
    """엔티티 데이터 클래스"""
    id: str
    name: str
    type: EntityType
    description: str = ""
    properties: Dict = field(default_factory=dict)
    source_chunks: List[str] = field(default_factory=list)

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "name": self.name,
            "type": self.type.value,
            "description": self.description,
            "properties": self.properties
        }

@dataclass
class Relationship:
    """관계 데이터 클래스"""
    id: str
    source: str  # Entity ID
    target: str  # Entity ID
    type: str
    description: str = ""
    weight: float = 1.0
    properties: Dict = field(default_factory=dict)

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "source": self.source,
            "target": self.target,
            "type": self.type,
            "description": self.description,
            "weight": self.weight
        }

@dataclass
class Triple:
    """트리플 (Subject, Predicate, Object)"""
    subject: str
    predicate: str
    object: str
    confidence: float = 1.0

    def __str__(self):
        return f"({self.subject}, {self.predicate}, {self.object})"


class KnowledgeGraphPipeline:
    """통합 Knowledge Graph 파이프라인"""
```

```python
    def __init__(self,
                 llm,
                 ner_model: str = "gliner",
                 embedding_model: str = "text-embedding-3-small"):
        self.llm = llm
        self.ner_model = ner_model
        self.embedding_model = embedding_model

        self.entities: Dict[str, Entity] = {}
        self.relationships: Dict[str, Relationship] = {}

        self._init_ner()

    def _init_ner(self):
        """NER 모델 초기화"""
        if self.ner_model == "gliner":
            from gliner import GLiNER
            self.ner = GLiNER.from_pretrained("urchade/gliner_multi-v2.1")
        elif self.ner_model == "spacy":
            import spacy
            self.ner = spacy.load("ko_core_news_lg")

    def process_document(self, document: dict) -> dict:
        """
        문서 처리 전체 파이프라인

        Args:
            document: {"id": "doc1", "content": "...", "metadata": {...}}

        Returns:
            {
                "entities": [...],
                "relationships": [...],
                "triples": [...],
                "stats": {...}
            }
        """
        content = document["content"]
        doc_id = document["id"]

        # 1. 엔티티 추출
        entities = self.extract_entities(content, doc_id)

        # 2. 관계 추출
        relationships = self.extract_relationships(content, entities, doc_id)

        # 3. 트리플 생성
        triples = self.generate_triples(entities, relationships)

        # 4. 엔티티 정규화 (중복 제거, 동의어 병합)
        entities = self.normalize_entities(entities)

        # 저장
        for e in entities:
```

```
            self.entities[e.id] = e
        for r in relationships:
            self.relationships[r.id] = r

        return {
            "entities": [e.to_dict() for e in entities],
            "relationships": [r.to_dict() for r in relationships],
            "triples": [str(t) for t in triples],
            "stats": {
                "entity_count": len(entities),
                "relationship_count": len(relationships),
                "triple_count": len(triples)
            }
        }

    def extract_entities(self, text: str, doc_id: str) -> List[Entity]:
        """엔티티 추출"""
        entities = []

        if self.ner_model == "gliner":
            labels = ["인물", "조직", "장소", "날짜", "제품", "이벤트"]
            raw_entities = self.ner.predict_entities(text, labels)

            for e in raw_entities:
                entity = Entity(
                    id=self._generate_id(e["text"]),
                    name=e["text"],
                    type=self._map_label_to_type(e["label"]),
                    source_chunks=[doc_id]
                )
                entities.append(entity)

        # LLM으로 추가 엔티티 추출 및 설명 생성
        entities = self._enrich_entities_with_llm(text, entities)

        return entities

    def _enrich_entities_with_llm(self, text: str, entities: List[Entity]) ->
List[Entity]:
        """LLM으로 엔티티 정보 보강"""
        entity_names = [e.name for e in entities]

        prompt = f"""
다음 텍스트와 추출된 엔티티를 분석하세요.

텍스트:
{text[:2000]}

추출된 엔티티: {entity_names}

각 엔티티에 대해 JSON 형식으로 설명을 생성하세요:
{{
    "entities": [
        {{
```

```
            "name": "엔티티명",
            "description": "이 텍스트에서의 역할과 특성 설명 (1-2문장)",
            "properties": {{"key": "value"}}
        }}
    ]
}}
"""
        response = self.llm.generate(prompt)
        enriched = json.loads(response)

        # 기존 엔티티에 설명 추가
        name_to_entity = {e.name: e for e in entities}
        for item in enriched.get("entities", []):
            if item["name"] in name_to_entity:
                name_to_entity[item["name"]].description = item["description"]
                name_to_entity[item["name"]].properties = item.get("properties",
{})

        return entities

    def extract_relationships(self,
                              text: str,
                              entities: List[Entity],
                              doc_id: str) -> List[Relationship]:
        """관계 추출 (LLM 기반)"""
        entity_names = [e.name for e in entities]

        prompt = f"""
다음 텍스트에서 엔티티 간 관계를 추출하세요.

텍스트:
{text[:3000]}

엔티티 목록: {entity_names}

관계 유형 예시:
- CEO_OF, FOUNDER_OF, EMPLOYEE_OF (인물 → 조직)
- LOCATED_IN, HEADQUARTERED_IN (조직/장소 → 장소)
- SUBSIDIARY_OF, PARTNER_OF, COMPETITOR_OF (조직 → 조직)
- OCCURRED_ON, STARTED_ON (이벤트 → 날짜)
- PRODUCES, DEVELOPS (조직 → 제품)
- CAUSES, LEADS_TO, RESULTS_IN (이벤트/개념 → 이벤트/개념)

JSON 형식으로 출력:
{{
    "relationships": [
        {{
            "source": "소스 엔티티",
            "target": "타겟 엔티티",
            "type": "관계유형",
            "description": "관계 설명",
            "weight": 0.0-1.0
        }}
    ]
```

```
}}
"""
        response = self.llm.generate(prompt)
        data = json.loads(response)

        relationships = []
        name_to_id = {e.name: e.id for e in entities}

        for r in data.get("relationships", []):
            if r["source"] in name_to_id and r["target"] in name_to_id:
                rel = Relationship(
                    id=self._generate_id(f"
{r['source']}_{r['type']}_{r['target']}"),
                    source=name_to_id[r["source"]],
                    target=name_to_id[r["target"]],
                    type=r["type"],
                    description=r.get("description", ""),
                    weight=r.get("weight", 1.0)
                )
                relationships.append(rel)

        return relationships

    def generate_triples(self,
                         entities: List[Entity],
                         relationships: List[Relationship]) -> List[Triple]:
        """트리플 생성"""
        id_to_name = {e.id: e.name for e in entities}
        triples = []

        for rel in relationships:
            triple = Triple(
                subject=id_to_name.get(rel.source, rel.source),
                predicate=rel.type,
                object=id_to_name.get(rel.target, rel.target),
                confidence=rel.weight
            )
            triples.append(triple)

        return triples

    def normalize_entities(self, entities: List[Entity]) -> List[Entity]:
        """엔티티 정규화 (중복 제거, 동의어 병합)"""
        # 간단한 정규화: 이름 기반 중복 제거
        seen = {}
        normalized = []

        for e in entities:
            # 정규화된 이름 (소문자, 공백 제거)
            norm_name = e.name.lower().replace(" ", "")

            if norm_name not in seen:
                seen[norm_name] = e
                normalized.append(e)
```

```python
            else:
                # 기존 엔티티에 소스 청크 병합
                seen[norm_name].source_chunks.extend(e.source_chunks)

        return normalized

    def _generate_id(self, text: str) -> str:
        """해시 기반 ID 생성"""
        return hashlib.md5(text.encode()).hexdigest()[:12]

    def _map_label_to_type(self, label: str) -> EntityType:
        """레이블을 EntityType으로 매핑"""
        mapping = {
            "인물": EntityType.PERSON,
            "조직": EntityType.ORGANIZATION,
            "장소": EntityType.LOCATION,
            "날짜": EntityType.DATE,
            "제품": EntityType.PRODUCT,
            "이벤트": EntityType.EVENT,
        }
        return mapping.get(label, EntityType.CONCEPT)

    def export_to_neo4j(self, uri: str, user: str, password: str):
        """Neo4j로 내보내기"""
        from neo4j import GraphDatabase

        driver = GraphDatabase.driver(uri, auth=(user, password))

        with driver.session() as session:
            # 엔티티 생성
            for entity in self.entities.values():
                session.run(
                    f"""
                    MERGE (e:{entity.type.value} {{id: $id}})
                    SET e.name = $name, e.description = $description
                    """,
                    id=entity.id,
                    name=entity.name,
                    description=entity.description
                )

            # 관계 생성
            for rel in self.relationships.values():
                session.run(
                    f"""
                    MATCH (a {{id: $source}})
                    MATCH (b {{id: $target}})
                    MERGE (a)-[r:{rel.type}]->(b)
                    SET r.description = $description, r.weight = $weight
                    """,
                    source=rel.source,
                    target=rel.target,
                    description=rel.description,
                    weight=rel.weight
```

```
                )

        driver.close()
        print(f"Exported {len(self.entities)} entities and
{len(self.relationships)} relationships to Neo4j")


# === 사용 예시 ===
from openai import OpenAI

class SimpleLLM:
    def __init__(self):
        self.client = OpenAI()

    def generate(self, prompt: str) -> str:
        response = self.client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": prompt}],
            response_format={"type": "json_object"}
        )
        return response.choices[0].message.content

# 파이프라인 실행
llm = SimpleLLM()
pipeline = KnowledgeGraphPipeline(llm=llm)

document = {
    "id": "news_001",
    "content": """
    삼성전자 이재용 회장이 2024년 CES에서 AI 반도체 전략을 발표했다.
    삼성전자는 NVIDIA와의 HBM 파트너십을 강화하고, 2025년까지 AI 칩 시장 점유율 20%를
목표로 한다.
    SK하이닉스 최태원 회장도 같은 행사에서 경쟁 제품을 공개했다.
    두 회사는 메모리 반도체 시장에서 치열한 경쟁을 벌이고 있다.
    """,
    "metadata": {"source": "news", "date": "2024-01-09"}
}

result = pipeline.process_document(document)

print("=== 추출 결과 ===")
print(f"\n엔티티 ({result['stats']['entity_count']}개):")
for e in result["entities"]:
    print(f"  - {e['name']} ({e['type']}): {e['description'][:50]}...")

print(f"\n관계 ({result['stats']['relationship_count']}개):")
for r in result["relationships"]:
    print(f"  - {r['source']} --[{r['type']}]--> {r['target']}")

print(f"\n트리플 ({result['stats']['triple_count']}개):")
for t in result["triples"]:
    print(f"  {t}")
```

실제 출력 예시:

```
=== 추출 결과 ===

엔티티 (8개):
  - 삼성전자 (ORG): 글로벌 반도체 및 전자제품 제조 대기업으로, AI 반도체 시장 진출...
  - 이재용 (PERSON): 삼성전자 회장으로 AI 반도체 전략을 주도하는 인물...
  - CES (EVENT): 2024년 라스베이거스에서 열린 세계 최대 가전/IT 전시회...
  - NVIDIA (ORG): 글로벌 GPU 및 AI 칩 선도 기업으로 HBM 파트너...
  - SK하이닉스 (ORG): 한국의 메모리 반도체 제조사로 삼성전자와 경쟁 관계...
  - 최태원 (PERSON): SK하이닉스(SK그룹) 회장...
  - 2024년 (DATE): CES 행사 및 발표가 이루어진 시점...
  - HBM (PRODUCT): High Bandwidth Memory, AI 연산에 필수적인 고성능 메모리...

관계 (7개):
  - 이재용 --[CEO_OF]--> 삼성전자
  - 최태원 --[CEO_OF]--> SK하이닉스
  - 삼성전자 --[PARTNER_OF]--> NVIDIA
  - 삼성전자 --[COMPETITOR_OF]--> SK하이닉스
  - 삼성전자 --[PRODUCES]--> HBM
  - SK하이닉스 --[PRODUCES]--> HBM
  - 이재용 --[PRESENTED_AT]--> CES

트리플 (7개):
  (이재용, CEO_OF, 삼성전자)
  (최태원, CEO_OF, SK하이닉스)
  (삼성전자, PARTNER_OF, NVIDIA)
  (삼성전자, COMPETITOR_OF, SK하이닉스)
  (삼성전자, PRODUCES, HBM)
  (SK하이닉스, PRODUCES, HBM)
  (이재용, PRESENTED_AT, CES)
```

---

13. Neo4j 통합 상세 가이드

13.1 Neo4j 설치 및 설정

```bash
# Docker로 Neo4j 실행
docker run -d \
    --name neo4j \
    -p 7474:7474 -p 7687:7687 \
    -e NEO4J_AUTH=neo4j/password123 \
    -v $HOME/neo4j/data:/data \
    neo4j:5.15.0

# Python 드라이버 설치
pip install neo4j
```

13.2 Neo4j 연결 및 기본 작업

```python
from neo4j import GraphDatabase
from typing import List, Dict, Any

class Neo4jKnowledgeGraph:
    """Neo4j 기반 Knowledge Graph"""

    def __init__(self, uri: str, user: str, password: str):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))

    def close(self):
        self.driver.close()

    def create_entity(self, entity: Dict) -> str:
        """엔티티 생성"""
        with self.driver.session() as session:
            result = session.run(
                f"""
                MERGE (e:{entity['type']} {{id: $id}})
                SET e.name = $name,
                    e.description = $description,
                    e.created_at = datetime()
                RETURN e.id as id
                """,
                id=entity["id"],
                name=entity["name"],
                description=entity.get("description", "")
            )
            return result.single()["id"]

    def create_relationship(self, rel: Dict) -> bool:
        """관계 생성"""
        with self.driver.session() as session:
            session.run(
                f"""
                MATCH (a {{id: $source}})
                MATCH (b {{id: $target}})
                MERGE (a)-[r:{rel['type']}]->(b)
                SET r.description = $description,
                    r.weight = $weight,
                    r.created_at = datetime()
                """,
                source=rel["source"],
                target=rel["target"],
                description=rel.get("description", ""),
                weight=rel.get("weight", 1.0)
            )
            return True

    def find_entity(self, name: str) -> Dict:
        """엔티티 검색"""
        with self.driver.session() as session:
            result = session.run(
```

```
                """
                MATCH (e)
                WHERE e.name CONTAINS $name
                RETURN e.id as id, e.name as name, labels(e)[0] as type,
                       e.description as description
                LIMIT 10
                """,
                name=name
            )
            return [dict(record) for record in result]

    def find_relationships(self, entity_id: str, depth: int = 1) -> Dict:
        """엔티티의 관계 탐색 (n-hop)"""
        with self.driver.session() as session:
            result = session.run(
                f"""
                MATCH path = (e {{id: $id}})-[*1..{depth}]-(related)
                RETURN e.name as source,
                       [r in relationships(path) | type(r)] as relationships,
                       [n in nodes(path)[1..] | n.name] as targets
                LIMIT 50
                """,
                id=entity_id
            )
            return [dict(record) for record in result]

    def get_subgraph(self, entity_ids: List[str]) -> Dict:
        """서브그래프 추출"""
        with self.driver.session() as session:
            result = session.run(
                """
                MATCH (e)
                WHERE e.id IN $ids
                OPTIONAL MATCH (e)-[r]-(related)
                WHERE related.id IN $ids
                RETURN collect(DISTINCT e) as nodes,
                       collect(DISTINCT r) as relationships
                """,
                ids=entity_ids
            )
            record = result.single()
            return {
                "nodes": [dict(n) for n in record["nodes"]],
                "relationships": [
                    {
                        "type": r.type,
                        "start": r.start_node["id"],
                        "end": r.end_node["id"]
                    }
                    for r in record["relationships"] if r
                ]
            }

    def run_pagerank(self) -> List[Dict]:
```

```
        """PageRank 실행 (중요 엔티티 식별)"""
        with self.driver.session() as session:
            # GDS 라이브러리 필요
            result = session.run(
                """
                CALL gds.pageRank.stream('myGraph')
                YIELD nodeId, score
                RETURN gds.util.asNode(nodeId).name AS name, score
                ORDER BY score DESC
                LIMIT 20
                """
            )
            return [dict(record) for record in result]

    def find_shortest_path(self, source_id: str, target_id: str) -> Dict:
        """최단 경로 탐색"""
        with self.driver.session() as session:
            result = session.run(
                """
                MATCH path = shortestPath(
                    (a {id: $source})-[*..10]-(b {id: $target})
                )
                RETURN [n in nodes(path) | n.name] as nodes,
                       [r in relationships(path) | type(r)] as relationships,
                       length(path) as distance
                """,
                source=source_id,
                target=target_id
            )
            record = result.single()
            if record:
                return dict(record)
            return None


# === 사용 예시 ===
kg = Neo4jKnowledgeGraph(
    uri="bolt://localhost:7687",
    user="neo4j",
    password="password123"
)

# 엔티티 생성
kg.create_entity({
    "id": "samsung_001",
    "type": "Organization",
    "name": "삼성전자",
    "description": "글로벌 전자제품 및 반도체 제조 기업"
})

kg.create_entity({
    "id": "person_001",
    "type": "Person",
    "name": "이재용",
```

```
    "description": "삼성전자 회장"
})

# 관계 생성
kg.create_relationship({
    "source": "person_001",
    "target": "samsung_001",
    "type": "CEO_OF",
    "description": "삼성전자의 회장으로 경영 총괄",
    "weight": 1.0
})

# 검색
entities = kg.find_entity("삼성")
print("검색 결과:", entities)

# 관계 탐색
relations = kg.find_relationships("samsung_001", depth=2)
print("관계:", relations)

# 최단 경로
path = kg.find_shortest_path("person_001", "nvidia_001")
print("경로:", path)

kg.close()
```

13.3 Cypher 쿼리 예시

```
// 1. 특정 타입의 모든 엔티티 조회
MATCH (p:Person)
RETURN p.name, p.description
LIMIT 20;

// 2. 특정 관계 패턴 검색
MATCH (p:Person)-[:CEO_OF]->(o:Organization)
RETURN p.name as CEO, o.name as Company;

// 3. 2-hop 관계 탐색
MATCH (start:Organization {name: "삼성전자"})-[*1..2]-(connected)
RETURN DISTINCT connected.name, labels(connected)[0] as type;

// 4. 공통 연결 찾기 (두 엔티티의 공통점)
MATCH (a:Organization {name: "삼성전자"})-[r1]-(common)-[r2]-(b:Organization
{name: "SK하이닉스"})
RETURN common.name, type(r1), type(r2);

// 5. 경쟁사 분석
MATCH (c1:Organization)-[:COMPETITOR_OF]-(c2:Organization)
WHERE c1.name = "삼성전자"
MATCH (c2)-[:PRODUCES]->(p:Product)
RETURN c2.name as Competitor, collect(p.name) as Products;
```

```
// 6. 시간순 이벤트 조회
MATCH (e:Event)-[:OCCURRED_ON]->(d:Date)
WHERE d.value >= date("2024-01-01")
RETURN e.name, d.value
ORDER BY d.value;

// 7. 영향력 있는 엔티티 (연결 수 기준)
MATCH (n)
WITH n, size((n)--()) as degree
ORDER BY degree DESC
LIMIT 10
RETURN n.name, labels(n)[0] as type, degree;

// 8. 클러스터 분석
CALL gds.louvain.stream('myGraph')
YIELD nodeId, communityId
WITH gds.util.asNode(nodeId) as node, communityId
RETURN communityId, collect(node.name) as members
ORDER BY size(members) DESC;
```

---

14. 그래프 시각화

14.1 NetworkX + Matplotlib

```python
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib import font_manager
import numpy as np

def visualize_knowledge_graph(
    entities: List[Dict],
    relationships: List[Dict],
    output_path: str = "kg_visualization.png"
):
    """Knowledge Graph 시각화"""

    # 한글 폰트 설정 (Windows)
    font_path = "C:/Windows/Fonts/malgun.ttf"
    font_prop = font_manager.FontProperties(fname=font_path)
    plt.rcParams['font.family'] = font_prop.get_name()

    # 그래프 생성
    G = nx.DiGraph()

    # 노드 추가
    node_colors = []
    color_map = {
        "PERSON": "#FF6B6B",
        "ORG": "#4ECDC4",
```

```
        "LOCATION": "#45B7D1",
        "PRODUCT": "#96CEB4",
        "EVENT": "#FFEAA7",
        "DATE": "#DDA0DD"
    }

    for e in entities:
        G.add_node(e["name"], **e)
        node_colors.append(color_map.get(e["type"], "#CCCCCC"))

    # 엣지 추가
    edge_labels = {}
    for r in relationships:
        source_name = next((e["name"] for e in entities if e["id"] ==
r["source"]), r["source"])
        target_name = next((e["name"] for e in entities if e["id"] ==
r["target"]), r["target"])
        G.add_edge(source_name, target_name, type=r["type"],
weight=r.get("weight", 1))
        edge_labels[(source_name, target_name)] = r["type"]

    # 레이아웃 계산
    pos = nx.spring_layout(G, k=2, iterations=50, seed=42)

    # 그리기
    plt.figure(figsize=(16, 12))

    # 노드
    nx.draw_networkx_nodes(
        G, pos,
        node_color=node_colors,
        node_size=2000,
        alpha=0.9
    )

    # 노드 라벨
    nx.draw_networkx_labels(
        G, pos,
        font_size=10,
        font_family=font_prop.get_name()
    )

    # 엣지
    nx.draw_networkx_edges(
        G, pos,
        edge_color='gray',
        arrows=True,
        arrowsize=20,
        alpha=0.6,
        connectionstyle="arc3,rad=0.1"
    )

    # 엣지 라벨
    nx.draw_networkx_edge_labels(
```

```
        G, pos,
        edge_labels=edge_labels,
        font_size=8,
        font_family=font_prop.get_name()
    )

    # 범례
    legend_elements = [
        plt.scatter([], [], c=color, s=100, label=label)
        for label, color in color_map.items()
    ]
    plt.legend(handles=legend_elements, loc='upper left', fontsize=10)

    plt.title("Knowledge Graph Visualization", fontsize=16)
    plt.axis('off')
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()

    print(f"시각화 저장: {output_path}")


# 사용 예시
visualize_knowledge_graph(
    entities=[
        {"id": "1", "name": "삼성전자", "type": "ORG"},
        {"id": "2", "name": "이재용", "type": "PERSON"},
        {"id": "3", "name": "NVIDIA", "type": "ORG"},
        {"id": "4", "name": "HBM", "type": "PRODUCT"},
    ],
    relationships=[
        {"source": "2", "target": "1", "type": "CEO_OF"},
        {"source": "1", "target": "3", "type": "PARTNER_OF"},
        {"source": "1", "target": "4", "type": "PRODUCES"},
    ]
)
```

14.2 Pyvis 인터랙티브 시각화

```
from pyvis.network import Network

def visualize_interactive(
    entities: List[Dict],
    relationships: List[Dict],
    output_path: str = "kg_interactive.html"
):
    """인터랙티브 Knowledge Graph 시각화"""

    # 네트워크 생성
    net = Network(
        height="800px",
```

```
        width="100%",
        bgcolor="#222222",
        font_color="white",
        directed=True
    )

    # 물리 설정
    net.set_options("""
    {
        "physics": {
            "forceAtlas2Based": {
                "gravitationalConstant": -50,
                "centralGravity": 0.01,
                "springLength": 200,
                "springConstant": 0.08
            },
            "solver": "forceAtlas2Based"
        }
    }
    """)

    # 색상 매핑
    color_map = {
        "PERSON": "#FF6B6B",
        "ORG": "#4ECDC4",
        "LOCATION": "#45B7D1",
        "PRODUCT": "#96CEB4",
        "EVENT": "#FFEAA7"
    }

    # 노드 추가
    for e in entities:
        net.add_node(
            e["id"],
            label=e["name"],
            title=f"{e['name']}\n{e.get('description', '')}",
            color=color_map.get(e["type"], "#CCCCCC"),
            size=30
        )

    # 엣지 추가
    for r in relationships:
        net.add_edge(
            r["source"],
            r["target"],
            title=r["type"],
            label=r["type"],
            arrows="to"
        )

    # HTML 저장
    net.save_graph(output_path)
    print(f"인터랙티브 시각화 저장: {output_path}")
```

```
# 사용
visualize_interactive(entities, relationships)
```

---

15. 평가 및 벤치마크

15.1 KG 품질 메트릭

```
from dataclasses import dataclass
from typing import List, Dict, Set, Tuple
import numpy as np

@dataclass
class KGQualityMetrics:
    """Knowledge Graph 품질 메트릭"""
    entity_precision: float
    entity_recall: float
    entity_f1: float
    relation_precision: float
    relation_recall: float
    relation_f1: float
    triple_accuracy: float
    graph_connectivity: float
    avg_node_degree: float

class KGEvaluator:
    """Knowledge Graph 평가기"""

    def evaluate(self,
                 predicted_entities: List[Dict],
                 predicted_relations: List[Dict],
                 gold_entities: List[Dict],
                 gold_relations: List[Dict]) -> KGQualityMetrics:
        """KG 품질 평가"""

        # 엔티티 평가
        pred_entity_set = {(e["name"], e["type"]) for e in predicted_entities}
        gold_entity_set = {(e["name"], e["type"]) for e in gold_entities}

        entity_precision, entity_recall, entity_f1 = self._calculate_prf(
            pred_entity_set, gold_entity_set
        )

        # 관계 평가
        pred_rel_set = {
            (r["source"], r["type"], r["target"])
            for r in predicted_relations
        }
        gold_rel_set = {
            (r["source"], r["type"], r["target"])
```

```
            for r in gold_relations
        }

        rel_precision, rel_recall, rel_f1 = self._calculate_prf(
            pred_rel_set, gold_rel_set
        )

        # 트리플 정확도 (완전 일치)
        triple_accuracy = len(pred_rel_set & gold_rel_set) / len(gold_rel_set) if
gold_rel_set else 0

        # 그래프 연결성
        connectivity = self._calculate_connectivity(predicted_entities,
predicted_relations)

        # 평균 노드 차수
        avg_degree = self._calculate_avg_degree(predicted_entities,
predicted_relations)

        return KGQualityMetrics(
            entity_precision=entity_precision,
            entity_recall=entity_recall,
            entity_f1=entity_f1,
            relation_precision=rel_precision,
            relation_recall=rel_recall,
            relation_f1=rel_f1,
            triple_accuracy=triple_accuracy,
            graph_connectivity=connectivity,
            avg_node_degree=avg_degree
        )

    def _calculate_prf(self, predicted: Set, gold: Set) -> Tuple[float, float,
float]:
        """Precision, Recall, F1 계산"""
        if not predicted:
            return 0.0, 0.0, 0.0

        true_positives = len(predicted & gold)
        precision = true_positives / len(predicted) if predicted else 0
        recall = true_positives / len(gold) if gold else 0
        f1 = 2 * precision * recall / (precision + recall) if (precision + recall)
> 0 else 0

        return precision, recall, f1

    def _calculate_connectivity(self,
                                entities: List[Dict],
                                relations: List[Dict]) -> float:
        """그래프 연결성 (연결된 노드 비율)"""
        if not entities:
            return 0.0

        connected_ids = set()
        for r in relations:
```

```
            connected_ids.add(r["source"])
            connected_ids.add(r["target"])

        entity_ids = {e["id"] for e in entities}
        return len(connected_ids & entity_ids) / len(entity_ids)

    def _calculate_avg_degree(self,
                              entities: List[Dict],
                              relations: List[Dict]) -> float:
        """평균 노드 차수"""
        if not entities:
            return 0.0

        degree_count = {}
        for r in relations:
            degree_count[r["source"]] = degree_count.get(r["source"], 0) + 1
            degree_count[r["target"]] = degree_count.get(r["target"], 0) + 1

        total_degree = sum(degree_count.values())
        return total_degree / len(entities)


# 사용 예시
evaluator = KGEvaluator()

predicted_entities = [
    {"id": "1", "name": "삼성전자", "type": "ORG"},
    {"id": "2", "name": "이재용", "type": "PERSON"},
]

gold_entities = [
    {"id": "1", "name": "삼성전자", "type": "ORG"},
    {"id": "2", "name": "이재용", "type": "PERSON"},
    {"id": "3", "name": "SK하이닉스", "type": "ORG"},
]

predicted_relations = [
    {"source": "2", "target": "1", "type": "CEO_OF"},
]

gold_relations = [
    {"source": "2", "target": "1", "type": "CEO_OF"},
    {"source": "1", "target": "3", "type": "COMPETITOR_OF"},
]

metrics = evaluator.evaluate(
    predicted_entities, predicted_relations,
    gold_entities, gold_relations
)

print("=== KG 품질 평가 결과 ===")
print(f"엔티티 Precision: {metrics.entity_precision:.3f}")
print(f"엔티티 Recall: {metrics.entity_recall:.3f}")
print(f"엔티티 F1: {metrics.entity_f1:.3f}")
```

```
print(f"관계 Precision: {metrics.relation_precision:.3f}")
print(f"관계 Recall: {metrics.relation_recall:.3f}")
print(f"관계 F1: {metrics.relation_f1:.3f}")
print(f"트리플 정확도: {metrics.triple_accuracy:.3f}")
print(f"그래프 연결성: {metrics.graph_connectivity:.3f}")
print(f"평균 노드 차수: {metrics.avg_node_degree:.2f}")
```

출력 예시:

```
=== KG 품질 평가 결과 ===
엔티티 Precision: 1.000
엔티티 Recall: 0.667
엔티티 F1: 0.800
관계 Precision: 1.000
관계 Recall: 0.500
관계 F1: 0.667
트리플 정확도: 0.500
그래프 연결성: 1.000
평균 노드 차수: 1.00
```

---

16. 트러블슈팅

16.1 일반적인 문제

문제	원인	해결책
엔티티 중복	동의어/약어 미처리	엔티티 정규화, 코레퍼런스 해결
관계 누락	암시적 관계	LLM 기반 관계 추론 추가
잘못된 관계	NLP 오류	Human-in-the-loop 검증
그래프 단절	문서 간 연결 부족	글로벌 엔티티 매칭
성능 저하	그래프 크기	인덱싱, 샤딩, 캐싱
메모리 부족	대규모 그래프	배치 처리, 스트리밍

16.2 엔티티 코레퍼런스 해결

```
def resolve_coreferences(text: str, entities: List[Entity]) -> List[Entity]:
    """
    대명사와 지시어를 엔티티로 해결

    예: "그는 삼성전자를 이끌고 있다" → "이재용은 삼성전자를 이끌고 있다"
    """
    # 간단한 규칙 기반 해결
    pronoun_map = {
        "그": None,  # 가장 최근 PERSON
```

```
        "그녀": None,
        "그것": None,
        "이 회사": None,  # 가장 최근 ORG
        "해당 기업": None,
    }

    # 최근 엔티티 추적
    last_person = None
    last_org = None

    for entity in entities:
        if entity.type == EntityType.PERSON:
            last_person = entity
        elif entity.type == EntityType.ORGANIZATION:
            last_org = entity

    # 대명사 해결
    resolved_text = text
    if last_person:
        resolved_text = resolved_text.replace("그는", f"{last_person.name}은")
        resolved_text = resolved_text.replace("그가", f"{last_person.name}이")

    if last_org:
        resolved_text = resolved_text.replace("이 회사는", f"{last_org.name}은")
        resolved_text = resolved_text.replace("해당 기업은", f"{last_org.name}은")

    return resolved_text
```

16.3 엔티티 연결 (Entity Linking)

```
def link_entities_to_kb(
    entities: List[Entity],
    knowledge_base: str = "wikidata"
) -> List[Entity]:
    """
    엔티티를 외부 지식베이스에 연결

    예: "삼성전자" → Wikidata Q20718
    """
    import requests

    for entity in entities:
        # Wikidata API 검색
        response = requests.get(
            "https://www.wikidata.org/w/api.php",
            params={
                "action": "wbsearchentities",
                "search": entity.name,
                "language": "ko",
                "format": "json"
            }
```

```
        )

        data = response.json()
        if data.get("search"):
            top_result = data["search"][0]
            entity.properties["wikidata_id"] = top_result["id"]
            entity.properties["wikidata_label"] = top_result.get("label", "")
            entity.properties["wikidata_description"] =
top_result.get("description", "")

    return entities
```

---

17. 참고 자료

17.1 공식 문서

- Microsoft GraphRAG
- Neo4j Documentation
- GLiNER
- spaCy

17.2 논문

- GraphRAG: Unlocking LLM Discovery on Narrative Private Data
- Docs2KG: Unified Knowledge Graph Construction from Heterogeneous Documents
- Knowledge Graphs

17.3 도구

- LangChain Graph
- LlamaIndex Knowledge Graph
- Weaviate - 벡터 + 그래프 하이브리드